Table of Contents

Comprehensive Question Bank: Unit 2 — Programming and Problem Solving with Python

Section A: Multiple-Choice Questions (MCQs)

Topic 2.1: Introduction to Python Programming

1. Who created the Python programming language?
   A) Dennis Ritchie
   B) Guido van Rossum
   C) James Gosling
   D) Bjarne Stroustrup

2. In which year did Guido van Rossum begin developing Python?
   A) 1991
   B) 1989
   C) 1995
   D) 2000

3. Python was named after:
   A) A snake species
   B) The comedy show *Monty Python's Flying Circus*
   C) A Dutch city
   D) The creator's pet

4. Computer programming is best defined as:
   A) Only writing text on a screen
   B) Creating a set of instructions that tells a computer how to perform a task
   C) Installing software on a computer
   D) Designing computer hardware

5. Which of the following is the correct order of the basic programming steps?
   A) Execute, Write Code, Compile/Interpret, Output
   B) Write Code, Compile/Interpret, Execute, Output
   C) Compile/Interpret, Output, Write Code, Execute
   D) Output, Execute, Write Code, Compile/Interpret

6. What is the role of an interpreter in Python?
   A) It permanently stores code on the hard disk
   B) It translates and executes code so the computer can understand it
   C) It designs the user interface
   D) It only formats code style

7. From where can Python be downloaded for free?
   A) https://www.microsoft.com/
   B) https://www.python.org/
   C) https://www.java.com/
   D) https://www.github.com/

8. During Python installation on Windows, which checkbox is essential to run Python easily from the command line?
   A) Uncheck "Add Python to PATH"
   B) Choose a different IDE
   C) Check "Add Python to PATH"
   D) Install only the IDE

9. What does IDE stand for?
   A) Internal Data Editor
   B) Integrated Development Environment
   C) Interpreted Development Engine
   D) Internet Design Environment

10. Which of the following is an example of an IDE used for writing Python code?
    A) Microsoft Word
    B) IDLE
    C) Adobe Photoshop
    D) VLC Media Player

Topic 2.2: Basic Python Syntax and Structure

1. Which function is used to display output on the screen in Python?
   A) `display()`
   B) `print()`
   C) `output()`
   D) `show()`

2. What symbol is used to start a single-line comment in Python?
   A) `//`
   B) `#`
   C) `<!--`
   D) `**`

3. A multi-line comment in Python is enclosed within:
   A) `# #`
   B) `""" """`
   C) `< >`
   D) `{ }`

4. What is the purpose of a comment in Python code?
   A) It increases program execution speed
   B) It provides explanations or notes for humans and is ignored by the interpreter
   C) It stores variable values
   D) It converts code into machine language

5. What will the following code output?

```
python
   print("This is my first page")
```

   A) `This is my first page` (without quotes, but code fails)
   B) **`This is my first page`**
   C) An error, because print needs two arguments
   D) Nothing, since it's a comment

6. In Python, text enclosed in double quotation marks is called a:
   A) Integer
   B) String
   C) Boolean
   D) Tuple

Topic 2.2.1: Variables, Data Types and Input/Output

1. A variable in Python is best described as:
   A) A fixed value that never changes
   B) A named storage location in memory that can hold and change a value
   C) A type of loop
   D) A built-in Python function

2. Which of the following is a valid variable name in Python?

   A) **`variable1`**

   B) `1variable`

   C) `variable-name`

   D) `variable name`

3. Which rule is TRUE about Python variable names?

   A) They can start with a digit

   B) They are case-sensitive

   C) They can contain spaces

   D) They can be the same as reserved keywords

4. Which of the following is an invalid Python variable name?

   A) `_score`

   B) `student_name`

   C) **`2nd_place`**

   D) `age17`

5. Are `age` and `Age` the same variable in Python?

   A) Yes, Python ignores case

   B) No, Python is case-sensitive so they are different variables

   C) Yes, but only inside functions

   D) It depends on the operating system

6. Which of these CANNOT be used as a Python variable name?

   A) `total_marks`

   B) `_temp`

   C) **`for`**

   D) `Marks2`

7. What is the output of the following code?

   ```
   python
   age = 25
   print("Age: ", age)
   ```

   A) **`Age:  25`**

   B) `25`

   C) `Age`

   D) `age`

8. Which data type would `price = 19.99` be automatically assigned?
   A) `int`
   B) `float`
   C) `str`
   D) `bool`

9. Which data type stores only `True` or `False`?
   A) `int`
   B) `str`
   C) `bool`
   D) `float`

10. Which function is used to check the data type of a variable?
    A) `datatype()`
    B) `type()`
    C) `kind()`
    D) `check()`

11. Which function is used to collect input from a user in Python?
    A) `get()`
    B) `input()`
    C) `read()`
    D) `scan()`

12. What data type does the `input()` function ALWAYS return, by default?
    A) `int`
    B) `str`
    C) `float`
    D) `bool`

13. Which function converts user input text into a whole number?
    A) `float()`
    B) `int()`
    C) `str()`
    D) `num()`

14. Which function converts user input text into a decimal number?
    A) `int()`
    B) `float()`
    C) `str()`
    D) `dec()`

15. What will happen if you run `int(input("Enter your age: "))` and the user types `"twenty"`?
    A) It will correctly convert to the number 20
    B) Python will raise a `ValueError`
    C) It will store `0`
    D) It will store the text as-is

Topic 2.3: Operators and Expressions

1. An expression in Python is defined as:
    A) A single number with no operators
    B) A combination of variables, operators, and values that produces a result
    C) A comment inside code
    D) A type of loop

2. What does the `**` operator perform in Python?
    A) Multiplication
    B) Exponentiation
    C) Floor division
    D) Modulus

3. What is the output of `10 // 3` in Python?
    A) `3.33`
    B) `3`
    C) `1`
    D) `30`

4. What is the output of `10 % 3` in Python?
    A) `3`
    B) `1`
    C) `3.33`
    D) `0`

5. What is the output of `10 ** 3`?
    A) `13`
    B) `30`
    C) `1000`
    D) `100`

6. Which operator category returns a Boolean (`True`/`False`) result?
   A) Arithmetic operators
   B) Comparison operators
   C) Assignment operators
   D) None of these

7. What does the `==` operator do in Python?
   A) Assigns a value to a variable
   B) Checks if two values are equal
   C) Checks if two values are not equal
   D) Performs floor division

8. What is the key difference between `=` and `==` in Python?
   A) They are identical and interchangeable
   B) `=` assigns a value, while `==` compares two values for equality
   C) `=` compares values, while `==` assigns values
   D) `==` is only used with strings

9. Which compound assignment operator is equivalent to `a = a + b`?
   A) `a -= b`
   B) `a += b`
   C) `a *= b`
   D) `a **= b`

10. Which logical operator returns `True` only if BOTH conditions are `True`?
    A) `or`
    B) `and`
    C) `not`
    D) `xor`

11. What does the `not` operator do?
    A) Combines two conditions
    B) Reverses/flips a Boolean value
    C) Compares two numbers
    D) Performs division

12. What is the output of the following code?

```
python
    x = True
    y = False
    print(x or y)
```

    A) `True`

B) `False`

C) `None`

D) Error

13. According to Python's operator precedence, which is evaluated FIRST?

A) Addition

B) Parentheses

C) Multiplication

D) Subtraction

14. According to operator precedence, which is evaluated immediately after parentheses?

A) Addition and subtraction

B) Exponentiation

C) Modulus

D) Assignment

15. What is the result of the expression `(3 + 2) * 4`?

A) `14`

B) `20`

C) `24`

D) `10`

16. What is the result of the expression `3 + 4 * 2`?

A) `14`

B) `11`

C) `10`

D) `24`

17. What is the value of `result` after executing `result = (3 + 4) * 2`?

A) `11`

B) `14`

C) `10`

D) `7`

Topic 2.4: Control Structures — Decision Making

1. What is the purpose of an `if` statement in Python?

A) To repeat a block of code

B) To run a block of code only when a condition is true

C) To define a function

D) To import a module

2. What symbol must appear at the end of an `if` statement line in Python?

   A) A semicolon `;`

   B) A colon `:`

   C) A period `.`

   D) A comma `,`

3. In Python, what defines which lines of code belong inside an `if` block?

   A) Curly braces `{}`

   B) Indentation

   C) Semicolons**

   D) Parentheses

4. What is the purpose of the `else` clause in an `if-else` statement?

   A) It always runs, regardless of the condition

   B) It runs only when the `if` condition is false

   C) It defines a new function

   D) It stops the program

5. Which of the following is the correct syntax for a short-hand if-else statement?

   A) `if condition: action_if_true else: action_if_false`

   B) `action_if_true if condition else action_if_false`

   C) `condition ? action_if_true : action_if_false;`

   D) `action_if_true else condition if action_if_false`

6. What is the output of the following code?

   ```
   python
   temperature = 15
   m = "It's a hot day" if (temperature > 30) else "It's not a hot day"
   print(m)
   ```

   A) `It's a hot day`

   B) `It's not a hot day`

   C) `15`

   D) Error

7. In an `if-elif-else` chain, how many blocks can execute at most for a single run?

   A) All of them, if their conditions are true

   B) Only one — the first condition found to be true

   C) Exactly two

   D) None, unless explicitly called

8. What is the output of the following code?

   ```
   python
   weather = "cloudy"
   ```

```
    if weather == "sunny":
        print("Wear sunglasses")
    elif weather == "rainy":
        print("Take an umbrella")
    else:
        print("Enjoy your day!")
```

A) `Wear sunglasses`

B) `Take an umbrella`

C) `Enjoy your day!`

D) No output

9. What is the output of the following code?

```
python
    temperature, humidity, wind_speed = 25, 60, 15
    print("Hot and humid" if temperature > 30 and humidity > 50 else
          "Warm and breezy" if temperature == 25 and wind_speed > 10 else
          "Cool and dry" if temperature < 20 and humidity < 30 else
          "Moderate")
```

A) Hot

B) Warm

C) Cool

D) Nothing

10. Which keyword is used in Python to check multiple alternative conditions after an initial `if`?

A) `elseif`

B) `elif`

C) `elseif:`

D) `else if`

Topic 2.4: Control Structures — Looping Constructs

1. A `while` loop in Python checks its condition:

A) After every iteration only

B) Before every iteration

C) Only once, at the very start

D) Never

2. What is an "infinite loop"?

   A) A loop that runs exactly 100 times

   B) A loop whose condition never becomes false, so it never stops

   C) A loop with no code inside it

   D) A syntax error in Python

3. In the following code, what is missing that could cause an infinite loop?

```python
number = 1
while number < 10:
    print(number)
```

   A) The `while` keyword

   B) An increment statement like `number += 1`

   C) A colon

   D) The `print()` function

4. A `for` loop in Python is best used when:

   A) You never know how many times to repeat

   B) You want to iterate once for each item in a sequence

   C) You want the loop to never execute

   D) You need to define a class

5. What will the following code print?

```python
friends = ["Ahmad", "Ali", "Hassan"]
for friend in friends:
    print("Hello", friend)
```

   A) `Hello friends`

   B) `Hello Ahmad`, `Hello Ali`, `Hello Hassan`, each on a new line

   C) `Hello Ahmad Ali Hassan`

   D) An error

6. Which built-in function is commonly used with a `for` loop to generate a sequence of numbers?

   A) `sequence()`

   B) `range()`

   C) `numbers()`

   D) `series()`

7. What values does `range(1, 6)` generate?
   A) `1, 2, 3, 4, 5, 6`
   B) `1, 2, 3, 4, 5`
   C) `0, 1, 2, 3, 4, 5`
   D) `2, 3, 4, 5, 6`

8. What is the output of the following code?

```
python
   number = 1
   while number < 10:
       print(number)
       number += 1
```

   A) Prints `1` through `10`
   B) Prints `1` through `9`
   C) Prints only `1`
   D) Causes an infinite loop

9. Which loop type is generally best suited for iterating through a known list of items, one at a time?
   A) `while` loop
   B) `for` loop
   C) `do-while` loop
   D) `repeat` loop

10. What does `number += 1` inside a `while` loop typically accomplish?
    A) It resets the loop
    B) It increases `number` by 1, helping the loop eventually reach its stopping condition
    C) It decreases `number` by 1
    D) It ends the program

Topic 2.5: Python Modules and Built-in Data Structures — Functions

1. What keyword is used to define a function in Python?
   A) `function`
   B) `def`
   C) `define`
   D) `func`

2. What is the purpose of a function in programming?

   A) To store a single fixed value permanently

   B) To encapsulate a reusable block of code that performs a specific task

   C) To create a new data type

   D) To comment out code

3. In the function definition `def greet(name):`, what is `name` called?

   A) A return value

   B) A parameter

   C) A method

   D) A module

4. Which keyword is used to send a value back from a function to the code that called it?

   A) `output`

   B) `return`

   C) `send`

   D) `yield only`

5. What is the output of the following code?

```
python
   def add(a, b):
       return a + b
   print(add(4, 5))
```

   A) `45`

   B) `9`

   C) `4 5`

   D) Error

6. What is a "default parameter" in a Python function?

   A) A parameter that must always be provided

   B) A parameter that is automatically used if the caller doesn't supply a value

   C) A parameter that can only be a string

   D) A parameter that stores the function's name

7. What is the output of the following code?

```
python
   def greet(name="Student"):
       return "Hello, " + name + "!"
   print(greet())
```

   A) `Hello, !`

   B) `Hello, Student!`

   C) Error, because no argument was passed

   D) `Hello, name!`

8. "Invoking" a function means:
   A) Writing the function definition for the first time
   B) Calling the function by name to perform its task
   C) Deleting the function
   D) Renaming the function

Topic 2.5: Modules, Libraries, and Packages

1. What is the purpose of the `import` keyword in Python?
   A) To define a new function
   B) To bring in code from a module or library so it can be used
   C) To delete a module
   D) To create a new variable

2. Which built-in Python library would you use to generate a random number?
   A) `statistics`
   B) `random`
   C) `datetime`
   D) `math_random`

3. Which built-in Python library provides functions for getting the current date and time?
   A) `random`
   B) `datetime`
   C) `statistics`
   D) `calendar_tools`

4. Which built-in Python library provides functions like `mean()` for numerical data analysis?
   A) `random`
   B) `statistics`
   C) `datetime`
   D) `numbers`

5. What is a "package" in Python?
   A) A single line of code
   B) A directory containing related modules
   C) A type of variable
   D) A built-in data type

6. In the example `from ecommerce import products`, what is `ecommerce`?
   A) A function
   B) A package
   C) A variable
   D) A data type

Topic 2.6: Built-in Data Structures — Lists

1. Which symbol is used to create a list in Python?
   A) `()`
   B) `[]`
   C) `{}`
   D) `<>`

2. What is the index of the FIRST item in a Python list?
   A) `1`
   B) `0`
   C) `-1`
   D) There is no fixed starting index

3. Given `fruits = ["Mango", "Apple", "Banana"]`, what does `fruits[1]` return?
   A) `Mango`
   B) `Apple`
   C) `Banana`
   D) An error

4. Which of the following best describes a Python list?
   A) An immutable, fixed collection of items
   B) An ordered, mutable (changeable) collection of items
   C) A collection that can only store numbers
   D) A single key-value pair

5. Which method adds an item to the END of a list?
   A) `add()`
   B) `append()`
   C) `insert_end()`
   D) `push()`

6. Which method removes the first occurrence of a specified item from a list?
   A) `delete()`
   B) `remove()`
   C) `pop_item()`
   D) `erase()`

7. Which method arranges a list's items in ascending order?
   A) `order()`
   B) `sort()`
   C) `arrange()`
   D) `ascend()`

8. Which method reverses the order of items in a list?

   A) `flip()`

   B) `reverse()`

   C) `invert()`

   D) `backward()`

9. What is the output of the following code?

```
python
   fruits = ["Mango", "Apple", "Banana"]
   fruits[0] = "Orange"
   fruits.append("Pineapple")
   print(fruits)
```

   A) `['Mango', 'Apple', 'Banana', 'Pineapple']`

   B) `['Orange', 'Apple', 'Banana', 'Pineapple']`

   C) `['Orange', 'Apple', 'Banana']`

   D) Error

10. What is the operation used to combine (join) two lists in Python?

    A) `combine()`

    B) `concat()`

    C) `+`

    D) `merge()`

11. What is the output of the following code?

```
python
   numbers = [1, 2, 3, 4, 5]
   slice = numbers[1:4]
   print(slice)
```

    A) `[1, 2, 3, 4]`

    B) `[2, 3, 4]`

    C) `[2, 3, 4, 5]`

    D) `[1, 2, 3]`

12. What is the output of the following code?

```
python
   students = ["Ahmed", "Sara", "Ali"]
   students.append("Hina")
   students.sort()
   print(students)
```

    A) `['Ahmed', 'Sara', 'Ali', 'Hina']`

B) `['Ahmed', 'Ali', 'Hina', 'Sara']`

C) `['Hina', 'Sara', 'Ali', 'Ahmed']`

D) `['Sara', 'Ali', 'Ahmed', 'Hina']`

Topic 2.6: Built-in Data Structures — Tuples

1. What is the primary difference between a list and a tuple?

   A) Lists use `()`, tuples use `[]`

   B) Lists are mutable; tuples are immutable

   C) Tuples can only store numbers

   D) There is no difference

2. Which symbol is used to create a tuple in Python?

   A) `[]`

   B) `()`

   C) `{}`

   D) `<>`

3. What is the output of the following code?

   ```
   python
   my_tuple = (1, 2, 3, "Hello", 4.5)
   print(my_tuple[3])
   ```

   A) `3`

   B) `Hello`

   C) `4.5`

   D) `1`

4. Which function returns the number of items in a tuple?

   A) `size()`

   B) `len()`

   C) `count()`

   D) `items()`

5. Why might a developer prefer a tuple over a list for storing a PIN code?

   A) Tuples use less memory than lists in all cases

   B) Tuples are immutable, protecting fixed data from accidental changes

   C) Tuples can be sorted while lists cannot

   D) Tuples support more data types than lists

Topic 2.6: Indexing and Slicing

1. What indexing system does Python use for sequences like lists, tuples, and strings?
   A) One-based indexing
   B) Zero-based indexing
   C) Reverse-based indexing
   D) Alphabetical indexing

2. In a list, what does the index `-1` refer to?
   A) The first element
   B) The last element
   C) An invalid index
   D) The second element

3. What is the correct syntax structure for slicing a sequence?
   A) `sequence(start, stop, step)`
   B) `sequence[start:stop:step]`
   C) `sequence{start;stop;step}`
   D) `sequence.slice(start-stop-step)`

4. Given `fruits = ["Apple", "Banana", "Cherry", "Date", "Elderberry"]`, what does `fruits[-1]` return?
   A) `Apple`
   B) `Elderberry`
   C) `Date`
   D) An error

5. Given `fruits = ["Apple", "Banana", "Cherry", "Date", "Elderberry"]`, what does `fruits[1:4]` return?
   A) `['Apple', 'Banana', 'Cherry']`
   B) `['Banana', 'Cherry', 'Date']`
   C) `['Banana', 'Cherry', 'Date', 'Elderberry']`
   D) `['Cherry', 'Date']`

6. In slicing notation `sequence[start:stop:step]`, is the `stop` index included in the result?
   A) Yes, always included
   B) No, the `stop` index is excluded
   C) Only for negative indices
   D) Only for strings

Topic 2.7: Modular Programming in Python

1. What is the primary goal of modular programming?
   A) To write the entire program in a single file for simplicity
   B) To divide a program into smaller, manageable, and reusable modules
   C) To avoid using functions
   D) To eliminate the need for comments

2. What does the check `if __name__ == "__main__":` accomplish in a Python script?
   A) It renames the script
   B) It ensures certain code only runs when the file is executed directly, not when imported
   C) It imports all standard libraries automatically
   D) It deletes unused variables

3. If `calculator.py` contains reusable functions and is imported into `main.py`, what is `calculator.py` acting as?
   A) A class
   B) A module
   C) A loop
   D) A variable

4. What is one major benefit of modular programming in large software projects?
   A) It forces all developers to work on the same file simultaneously
   B) It allows different developers to work on different parts of a program independently
   C) It removes the need for testing
   D) It prevents code reuse

Topic 2.8: Object-Oriented Programming in Python

1. In Object-Oriented Programming, a class is best described as:
   A) A specific instance of data
   B) A blueprint or template that defines the structure and behavior of objects
   C) A type of loop
   D) A single variable

2. What is an "object" in Object-Oriented Programming?
   A) The same thing as a class
   B) A specific instance created from a class, with its own data
   C) A built-in Python function
   D) A comment in the code

3. Which special method in a Python class is automatically called when a new object is created?

   A) `__main__`

   B) `__init__`

   C) `__create__`

   D) `__start__`

4. In a Python class method, what does the parameter `self` refer to?

   A) The class definition itself, in general

   B) The specific object on which the method is being called

   C) A global variable

   D) The Python interpreter

5. Given the class below:

   ```python
   class ToyCar:
       def __init__(self, color, size, wheels):
           self.color = color
           self.size = size
           self.wheels = wheels
   car1 = ToyCar("red", "small", 4)
   ```

   What does `car1` represent?

   A) The class blueprint

   B) An object (instance) of the `ToyCar` class

   C) A Python module

   D) A tuple

6. If `car1` and `car2` are both objects of the same class, and you change `car1.color`, what happens to `car2.color`?

   A) It automatically changes to match `car1.color`

   B) It remains unchanged, since each object has independent data

   C) It causes a program crash

   D) It becomes `None`

7. A method defined inside a class, such as `describe()`, is best described as:

   A) A variable belonging to the class

   B) A function that belongs to the class and defines its behavior

   C) A comment inside the class

   D) A built-in Python keyword

Topic 2.9: Advanced Python Concepts — Exception Handling

1. What is an "exception" in Python?
   A) A comment that Python ignores
   B) An error that occurs during program execution
   C) A special type of variable
   D) A type of loop

2. What is the purpose of a `try-except` block?
   A) To make the program run faster
   B) To handle errors gracefully instead of letting the program crash
   C) To define a new function
   D) To create a new class

3. Which specific exception is raised when a program attempts to divide a number by zero?
   A) `ValueError`
   B) `ZeroDivisionError`
   C) `TypeError`
   D) `IndexError`

4. In a `try-except` block, what type of code should be placed inside the `try` block?
   A) Code that will never cause an error
   B) Code that might cause an error
   C) Only comments
   D) Only variable declarations

5. What is the most appropriate mindset toward reading Python error messages, according to good programming practice?
   A) Errors should be ignored, since they don't affect the program
   B) Errors should be read carefully, as they are honest status reports showing exactly where a problem occurred
   C) Errors mean the programmer should give up
   D) Errors only happen to beginners

Topic 2.9: Advanced Python Concepts — File Handling

1. Which function is used to open a file in Python?
   A) `file()`
   B) `open()`
   C) `load()`
   D) `access()`

2. Which file mode is used to read the contents of an existing file?
   A) `"w"`
   B) `"r"`
   C) `"a"`
   D) `"x"`
3. Which file mode overwrites all existing content in a file with new data?
   A) `"r"`
   B) `"w"`
   C) `"a"`
   D) `"n"`
4. Which file mode adds new content to a file WITHOUT deleting existing content?
   A) `"r"`
   B) `"w"`
   C) `"a"`
   D) `"d"`
5. What is the main benefit of using the `with` statement when working with files in Python?
   A) It makes the file permanently read-only
   B) It automatically closes the file safely, even if an error occurs
   C) It automatically deletes the file after use
   D) It converts the file to a different format
6. Which method is used to read the entire contents of a file into a variable?
   A) `file.get()`
   B) `file.read()`
   C) `file.load()`
   D) `file.open()`
7. Which method is used to write text into an already-open file?
   A) `file.print()`
   B) `file.write()`
   C) `file.output()`
   D) `file.save()`

Topic 2.10: Testing and Debugging in Python

1. What is the main purpose of "testing" in software development?
   A) To make code visually appealing
   B) To run code with various inputs to check whether it behaves as expected
   C) To permanently delete faulty code
   D) To translate code into another programming language

2. Which type of testing focuses on individual functions or classes in isolation?
   A) Integration Testing
   B) Unit Testing
   C) Functional Testing
   D) Regression Testing

3. Which type of testing checks whether different parts of a program work correctly together?
   A) Unit Testing
   B) Integration Testing
   C) Regression Testing
   D) Load Testing

4. Which type of testing ensures that new changes do not break previously working functionality?
   A) Unit Testing
   B) Functional Testing
   C) Regression Testing
   D) Integration Testing

5. Which type of testing validates software from the end user's perspective?
   A) Unit Testing
   B) Functional Testing
   C) Regression Testing
   D) Integration Testing

6. Which Python module is commonly used for unit testing?
   A) `testcase`
   B) `unittest`
   C) `pytester`
   D) `debug`

7. What is "debugging" in programming?
   A) Writing new features for a program
   B) The process of finding and fixing errors (bugs) in code
   C) Deleting a program entirely
   D) Compiling code into machine language

8. Who is credited with popularizing the term "debugging" after finding a literal moth in the Harvard Mark II computer?
   A) Ada Lovelace
   B) Grace Hopper
   C) Guido van Rossum
   D) Margaret Hamilton
9. Which of the following is a common, simple debugging technique?
   A) Deleting the entire program and starting over
   B) Adding print statements to check variable values at different points
   C) Ignoring all error messages
   D) Renaming all variables randomly
10. What is the purpose of Python's `pdb` module?
    A) To format printed output
    B) To step through code, pause execution, and inspect variables while debugging
    C) To generate random test data
    D) To manage package installations

Section B: Short Answer Questions (SQs)

Topic 2.1: Introduction to Python Programming

1. Define computer programming in your own words.
2. List the four basic steps involved in writing and running a computer program.
3. Explain briefly why Python is considered a good language for beginners.
4. State the purpose of a development environment.
5. Explain briefly why checking "Add Python to PATH" during installation is important.
6. Differentiate between Python itself and an IDE.

Topic 2.2: Basic Python Syntax and Structure

1. Define the term "syntax" as it applies to a programming language.
2. Explain briefly the purpose of comments in Python code.
3. Differentiate between a single-line comment and a multi-line comment in Python.
4. Identify the function used to display output on the screen and briefly explain what it does.
5. Predict the output of: `print("Python is fun")`.

Topic 2.2.1: Variables, Data Types and Input/Output

1. Define the term "variable" in Python.
2. State the four variable naming rules in Python.
3. Differentiate between `int`, `float`, `str`, and `bool` data types, giving one example of each.
4. Explain briefly why `age` and `Age` are treated as two different variables in Python.
5. Explain the difference between the `input()` function and the `print()` function.
6. Explain briefly why `int()` or `float()` conversion is needed when working with numeric input from a user.
7. Predict the output of the following code:

```python
name = "Sara"
print("Hello, " + name + "!")
```

8. Identify which of the following variable names are invalid and explain why: `2nd_student`, `student_name`, `class`.

Topic 2.3: Operators and Expressions

1. Define the term "expression" in Python, with an example.
2. Differentiate between the `/` and `//` arithmetic operators, giving an example of each.
3. Explain briefly the difference between `=` and `==` in Python.
4. List the three basic logical operators used in Python.
5. State Python's order of operator precedence from highest to lowest.
6. Predict the output of: `print(10 % 3)`.
7. Predict the output of: `print((3 + 4) * 2)`.
8. Explain briefly what a compound assignment operator like `+=` does.

Topic 2.4: Control Structures

1. Define the purpose of an `if` statement.
2. Differentiate between an `if-else` statement and an `if-elif-else` statement.
3. Explain briefly what a short-hand if-else statement is and when it might be useful.
4. Differentiate between a `while` loop and a `for` loop.
5. Explain briefly what an "infinite loop" is and how it can happen accidentally.
6. Map out, in words, the step-by-step execution flow of the following loop:

```python
number = 1
```

```
   while number < 4:
       print(number)
       number += 1
```

7. Explain briefly the role of the `range()` function in a `for` loop.
8. Predict the output of:

```
python
   for i in range(2, 5):
       print(i)
```

Topic 2.5: Functions, Modules, and Libraries

1. Define what a function is in Python.
2. Differentiate between defining a function and invoking (calling) a function.
3. Explain briefly the difference between a function parameter and a function's return value.
4. Explain briefly what a default parameter is, with an example.
5. Define what a module is in Python.
6. Differentiate between a module and a package.
7. State the purpose of the `import` keyword.
8. Explain briefly why libraries and modules save programmers time and effort.

Topic 2.6: Built-in Data Structures (Lists, Tuples, Indexing, Slicing)

1. Define what a list is in Python.
2. Differentiate between a list and a tuple, focusing on mutability.
3. List four built-in list methods and briefly state what each one does.
4. Explain briefly what zero-based indexing means.
5. Explain briefly the difference between indexing and slicing.
6. Explain briefly how negative indexing works, using an example.
7. Predict the output of the following code:

```
python
   numbers = [10, 20, 30, 40, 50]
   print(numbers[-2])
```

8. Explain briefly why a developer might choose a tuple instead of a list for storing fixed data such as coordinates.

Topic 2.7: Modular Programming

1. Define modular programming in your own words.
2. Explain briefly the purpose of the `if __name__ == "__main__":` construct.

3. Explain briefly one advantage of splitting a large program into multiple modules.
4. Differentiate between a script that is run directly and one that is imported as a module.

Topic 2.8: Object-Oriented Programming

1. Define the terms "class" and "object," and briefly explain how they relate to each other.
2. Explain briefly the purpose of the `__init__` method in a Python class.
3. Explain briefly what `self` refers to inside a class method.
4. Differentiate between an attribute and a method within a class.
5. Explain briefly why two objects created from the same class can hold different data.

Topic 2.9: Exception Handling and File Handling

1. Define the term "exception" in Python.
2. Explain briefly the purpose of a `try` block and an `except` block.
3. Identify the specific exception type raised when dividing a number by zero.
4. Explain briefly the difference between file mode `"w"` and file mode `"a"`.
5. Explain briefly why the `with` statement is recommended when working with files.
6. State the two main file operations covered in this unit (reading and writing) and briefly describe each.

Topic 2.10: Testing and Debugging

1. Define the term "testing" in software development.
2. Differentiate between unit testing and integration testing.
3. Explain briefly what regression testing checks for.
4. Define the term "debugging."
5. List two common debugging techniques mentioned in this unit.
6. Explain briefly why reading error messages carefully is an important debugging skill.

Section C: Long / Extensive Questions (LQs)

Topic 2.1–2.2: Programming Basics and Python Syntax

1. Discuss in detail the complete process of computer programming, from writing code to producing output.
   a) Describe each of the four basic programming steps (Write Code, Compile/Interpret, Execute, Output) and explain what happens at each stage.
   b) Explain the role Python's interpreter plays in this process, compared to a compiled

language.
c) Justify why Python's emphasis on readable syntax makes it a suitable first language for beginners.

2. Elaborate on the process of setting up a Python development environment and writing a first program.
a) Describe, step by step, how to install Python and correctly configure it for command-line use.
b) Explain the purpose of an IDE and compare at least two IDE options a student might use.
c) Construct a short "Hello World"-style program and explain, line by line, what each part does.

Topic 2.2.1–2.3: Variables, Data Types, and Operators

1. Analyze the concept of variables and data types in Python in comprehensive detail.
a) Explain what a variable is and how Python allocates and updates memory when a variable's value changes.
b) Discuss all four Python naming rules, providing at least two valid and two invalid variable name examples for each rule where relevant.
c) Compare and contrast the four core data types (`int`, `float`, `str`, `bool`), including how Python automatically determines type.

2. Evaluate the role of operators and operator precedence in constructing correct Python expressions.
a) Construct a table comparing arithmetic, comparison, assignment, and logical operators, including at least three examples of each.
b) Trace, step by step, how Python would evaluate the expression `(10 + 3) * (2 ** (2 - 1)) / 5`, showing the precedence order applied at each step.
c) Discuss a real-world scenario where confusing `=` with `==` could cause a serious logical bug in a program.

Topic 2.4: Control Structures

1. Design and analyze a complete decision-making system using Python conditional statements.
a) Construct an `if-elif-else` chain that classifies a student's exam score into "A", "B", "C", or "F" grades, and explain your reasoning for each condition.
b) Compare and contrast the regular `if-else` statement with the short-hand if-else statement, discussing when each is more appropriate.
c) Discuss how condition order affects the outcome in an `if-elif-else` chain, using a specific example where reordering conditions changes the result.

2. Construct and trace the execution of loop-based Python programs in detail.
a) Design a `while` loop that prints all even numbers from 1 to 50, and construct a trace table showing at least the first five iterations (variable values and condition checks).

b) Design an equivalent `for` loop using `range()` that accomplishes the same task, and discuss the differences in approach between the `while` and `for` versions.
c) Analyze what would happen if the increment statement were accidentally removed from your `while` loop, and explain how a programmer would detect and fix this bug.

Topic 2.5: Functions, Modules, and Libraries

1. Discuss in detail the role of functions and modules in writing efficient, maintainable Python code.
   a) Construct a function `calculate_area(shape, *dimensions)`-style design (in plain terms) that could compute the area of different shapes, and explain the role of parameters and return values in your design.
   b) Explain the difference between required parameters and default parameters, constructing an example of each within a single function.
   c) Evaluate the benefits of using Python's standard library (e.g., `random`, `datetime`, `statistics`) instead of writing equivalent functionality from scratch.
2. Analyze the concept of modular programming and package structure in larger Python projects.
   a) Design a package structure (naming at least three modules) for a school management system, briefly describing what each module would contain.
   b) Discuss how the `import` statement and package structure work together to keep large codebases organized.
   c) Justify why professional software teams almost always use modular design rather than writing an entire application in a single file.

Topic 2.6: Built-in Data Structures

1. Conduct a comprehensive comparative analysis of Python's lists and tuples.
   a) Construct a feature-by-feature comparison table covering mutability, syntax, common use cases, and available methods for lists versus tuples.
   b) Design a short Python program that creates a list of five student names, then demonstrates at least four different list operations (adding, modifying, sorting, removing).
   c) Evaluate a scenario where a program stores GPS coordinates: justify, with reasoning, whether a list or a tuple is the better data structure choice, and discuss the risk of choosing incorrectly.
2. Analyze indexing and slicing as tools for working with Python sequences.
   a) Explain, with a labeled diagram or index map, how positive and negative indices relate to each other on a five-item list.
   b) Construct at least four different slicing expressions (using varying start, stop, and step values) on a sample list, and state the expected output of each.
   c) Discuss why understanding zero-based indexing is essential before working with more advanced data structures later in the course.

Topic 2.7–2.8: Modular Programming and Object-Oriented Programming

1. Discuss in detail how modular programming and object-oriented programming both aim to manage complexity in software design.
   a) Compare and contrast modular programming (functions and modules) with object-oriented programming (classes and objects) as two different ways of organizing code.
   b) Design a `Student` class with attributes (name, roll number, marks) and at least one method that calculates or displays information about the student, explaining each part of your design.
   c) Evaluate why real-world software systems, such as a banking application, might use both modular programming and object-oriented programming together rather than relying on only one approach.

2. Analyze the concept of classes and objects using a real-world modeling example.
   a) Using the `ToyCar` class example structure, design a new class called `Library Book` with at least three attributes and two methods, explaining the purpose of each.
   b) Discuss the role of the `__init__` method and the `self` keyword in ensuring that each created object maintains independent data.
   c) Construct a short scenario demonstrating that changing one object's attribute does not affect another object of the same class, and explain why this happens at a conceptual level.

Topic 2.9: Exception Handling and File Handling

1. Evaluate the importance of exception handling in building robust, real-world Python applications.
   a) Design a `try-except` block that safely handles both a `ZeroDivisionError` and a `ValueError` within the same program, explaining the purpose of each `except` clause.
   b) Discuss why treating error messages as "helpful clues" rather than failures leads to better debugging outcomes for beginner programmers.
   c) Analyze a real-world scenario — such as a banking ATM program — where unhandled exceptions could cause serious consequences, and propose how exception handling could prevent them.

2. Discuss in detail the complete process of file handling in Python, from creation to retrieval of data.
   a) Construct a short program that writes three lines of student names into a file using write mode, then appends two more names using append mode.
   b) Explain the purpose of the `with` statement in file handling, and discuss what could go wrong if a file were opened without it.
   c) Design a follow-up program that reads the entire file back and displays its contents, discussing why file handling is essential for programs that need to persist data beyond a single run.

Topic 2.10: Testing and Debugging

1. Analyze the role of testing and debugging in the professional software development lifecycle.
   a) Construct a comparison table of the four testing types discussed in this unit (Unit, Integration, Functional, Regression), including what each one checks and a real-world example of each.
   b) Discuss, using the story of Grace Hopper and the Harvard Mark II moth, how the concept of a "bug" has evolved into modern debugging terminology.
   c) Evaluate at least three common debugging techniques (print statements, debugging tools, reading error messages) and justify which one you would use first when facing an unfamiliar bug, and why.
2. Design and trace a complete debugging scenario for a Python program containing multiple hidden errors.
   a) Construct a short buggy Python program (5–8 lines) containing at least two distinct bugs (e.g., a typo and a potential division by zero).
   b) Trace, step by step, how a programmer would use error messages to identify and locate each bug.
   c) Discuss how combining exception handling (Topic 2.9) with careful debugging practices creates more resilient, real-world-ready software.